# Fișa recapitulativă VIP: Transformers și Modele Lingvistice Mari

Afshine Amidi și Shervine Amidi

13 septembrie 2026

*Această fișă recapitulativă VIP oferă o privire de ansamblu asupra conținutului cărții „Super Study Guide: Transformers & Large Language Models", care conține ∼600 de ilustrații pe parcursul a 250 de pagini și aprofundează următoarele concepte. Mai multe detalii găsiți la* `https://superstudy.guide`.

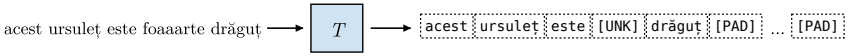
## 1 Fundamente

### 1.1 Tokeni

❐ **Definiție** – Un *token* este o unitate indivizibilă de text, precum un cuvânt, un subcuvânt sau un caracter, și face parte dintr-un vocabular predefinit.

*Remarcă: Tokenul necunoscut* `[UNK]` *reprezintă fragmente de text necunoscute, în timp ce tokenul de completare* `[PAD]` *este folosit pentru a umple pozițiile goale și a asigura lungimi consistente ale secvențelor de intrare.*

❐ **Tokenizator** – Un *tokenizator* $T$ împarte textul în tokeni, cu un nivel arbitrar de granularitate.




Iată principalele tipuri de tokenizatori:

| Tip | Avantaje | Dezavantaje | Ilustrație |
|---|---|---|---|
| Cuvânt | • Ușor de interpretat<br>• Secvență scurtă | • Dimensiune mare a vocabularului<br>• Variațiile cuvintelor nu sunt tratate | `ursulet` `drăguț` |
| Subcuvânt | • Rădăcinile cuvintelor sunt valorificate<br>• Embedding-uri intuitive | • Lungime crescută a secvenței<br>• Tokenizare mai complexă | `urs` `##uleț` `drăguț` |
| Caracter<br>Octet | • Fără probleme cu cuvintele din afară vocabularului<br>• Dimensiune mică a vocabularului | • Lungime mult mai mare a secvenței<br>• Tipare greu de interpretat, deoarece nivelul este prea scăzut | `u r s u l e ț`<br>`d r ă g u ț` |

*Remarcă: Byte-Pair Encoding (BPE) și Unigram sunt tokenizatori la nivel de subcuvânt folosiți frecvent.*

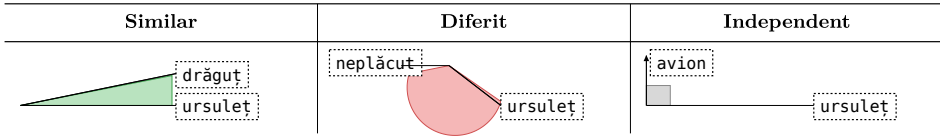
### 1.2 Embedding-uri

❐ **Definiție** – Un *embedding* (încorporare) este o reprezentare numerică a unui element (de ex. token, propoziție) și este caracterizat de un vector $\boldsymbol{x} \in \mathbb{R}^n$.

❐ **Similaritate** – *Similaritatea cosinus* dintre doi tokeni $t_1, t_2$ este dată de:

$$\text{similaritate}(t_1, t_2) = \frac{t_1 \cdot t_2}{||t_1||\,||t_2||} = \cos(\theta) \in [-1, 1]$$

Unghiul $\theta$ caracterizează similaritatea dintre cei doi tokeni:

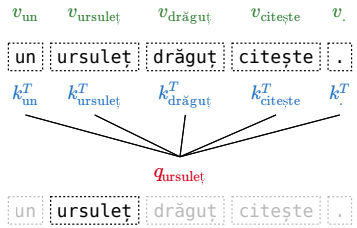



*Remarcă: Căutarea celor mai apropiați vecini aproximativi (Approximate Nearest Neighbors, ANN) și hashing-ul sensibil la localitate (Locality Sensitive Hashing, LSH) sunt metode care aproximează eficient operația de similaritate pe baze de date mari.*

## 2 Transformers

### 2.1 Atenție

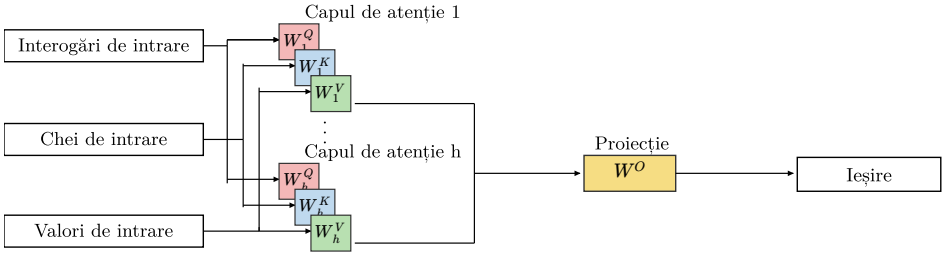
❐ **Formula** – Dat fiind un *query* (o interogare) $\boldsymbol{q}$, dorim să aflăm căreia *cheie* $\boldsymbol{k}$ ar trebui să îi acorde interogarea „atenție", în raport cu *valoarea* $\boldsymbol{v}$ asociată.




Atenția poate fi calculată eficient cu ajutorul matricelor $Q, K, V$, care conțin interogările $\boldsymbol{q}$, cheile $\boldsymbol{k}$, respectiv valorile $\boldsymbol{v}$, împreună cu dimensiunea $d_k$ a cheilor:

$$\text{atenție} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

❐ **MHA** – Un strat de *Atenție Multi-Cap* (***M**ulti-**H**ead **A**ttention*, MHA) efectuează calcule de atenție pe mai multe capete, apoi proiectează rezultatul în spațiul de ieșire.

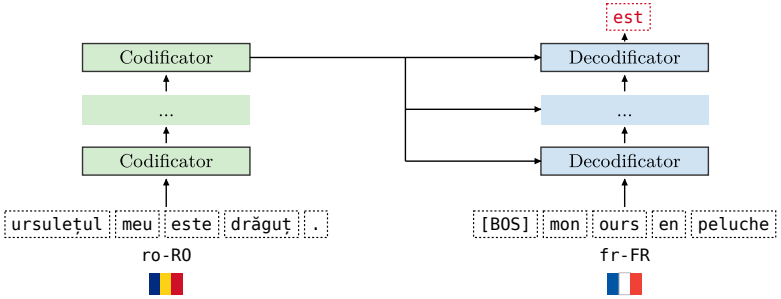



Este compus din $h$ capete de atenție, precum și din matricele $W^Q, W^K, W^V$, care proiectează intrarea pentru a obține interogările $Q$, cheile $K$ și valorile $V$. Proiecția se realizează cu ajutorul matricei $W^O$.

*Remarcă: Atenția cu interogări grupate (Grouped-Query Attention, GQA) și atenția multi-interogare (Multi-Query Attention, MQA) sunt variante ale MHA care reduc costul computațional prin partajarea cheilor și a valorilor între capetele de atenție.*

### 2.2 Arhitectura

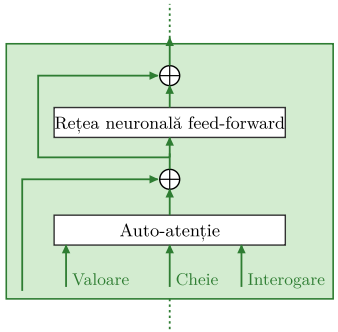
❒ **Privire de ansamblu** – *Transformer*-ul este un model de referință care se bazează pe mecanismul de auto-atenție și este compus din codificatoare și decodificatoare. Codificatoarele calculează embedding-uri semnificative ale intrării, care sunt apoi folosite de decodificatoare pentru a prezice următorul token din secvență.

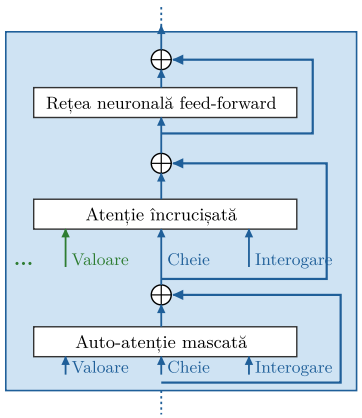



*Remarcă: Deși Transformer-ul a fost propus inițial ca model pentru sarcini de traducere, în prezent este folosit pe scară largă în multe alte aplicații.*

❒ **Componente** – *Codificatorul* și *decodificatorul* sunt două componente fundamentale ale Transformer-ului și au roluri diferite:

| Codificator | Decodificator |
|---|---|
| Embedding-urile codificate încapsulează sensul intrării | Embedding-urile decodificate încapsulează sensul atât al intrării, cât și al ieșirii prezise până acum |



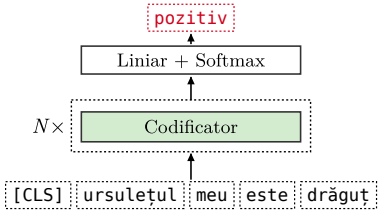




❒ **Embedding-uri poziționale** – *Embedding-urile poziționale* indică poziția tokenului în secvență. Pot fi definite arbitrar sau învățate din date.

*Remarcă: Rotary Position Embeddings (RoPE), adică embedding-uri poziționale rotative, sunt o variantă populară și eficientă, care rotește vectorii de interogare și de cheie pentru a încorpora informații despre poziția relativă.*

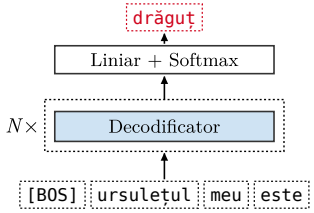
### 2.3 Variante

❒ **Doar codificator** – ***B**idirectional **E**ncoder **R**epresentations from **T**ransformers* (BERT) este un model bazat pe Transformer, compus dintr-o stivă de codificatoare, care primește un text la intrare și produce embedding-uri semnificative, ce pot fi folosite ulterior în sarcini de clasificare.




Un token `[CLS]` este adăugat la începutul secvenței pentru a capta sensul propoziției. Embedding-ul său codificat este adesea folosit în sarcini ulterioare, precum analiza sentimentului.

❒ **Doar decodificator** – ***G**enerative **P**re-trained **T**ransformer* (GPT) este un model autoregresiv bazat pe Transformer, compus dintr-o stivă de decodificatoare. Spre deosebire de BERT și derivatele sale, GPT tratează toate problemele ca probleme de tip text-la-text.




Majoritatea LLM-urilor de ultimă generație se bazează pe o arhitectură doar cu decodificator, precum seria GPT, DeepSeek, Mistral, Gemma etc.

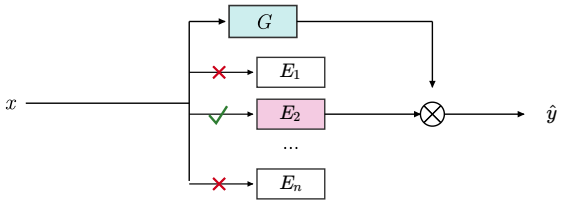
*Remarcă: Modelele codificator-decodificator, precum T5, sunt de asemenea autoregresive și au multe caracteristici comune cu modelele doar cu decodificator.*

## 3 Modele lingvistice mari

### 3.1 Privire de ansamblu

❐ **Definiție** – Un **Model Lingvistic Mare** (***L**arge **L**anguage **M**odel*, LLM) este un model bazat pe Transformer, cu capacități NLP puternice. Este „mare” în sensul că de obicei conține miliarde de parametri și este antrenat pe o cantitate mare de date.

❐ **Amestec de experți** – Un **Amestec de Experți** (***M**ixture of **E**xperts*, MoE) este un model care activează doar o parte din neuronii săi la momentul inferenței. Se bazează pe o poartă (*gate*) $G$ și pe experții $E_1, ..., E_n$.




$$\hat{y} = \sum_{i=1}^{n} G(x)_i E_i(x)$$

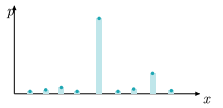
LLM-urile bazate pe MoE folosesc acest mecanism de tip poartă în rețelele lor FFNN pentru a direcționa fiecare token către un subset de experți.

### 3.2 Promptare

❐ **Lungimea contextului** – *Lungimea contextului* unui model este numărul maxim de tokeni care pot încăpea în intrare. De obicei variază de la zeci de mii până la milioane de tokeni.

*Remarcă: Calitatea răspunsurilor unui LLM tinde să scadă pe măsură ce fereastra de context se umple, fenomen cunoscut sub numele de „context rot”.*

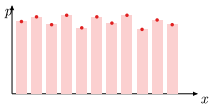
❐ **Eșantionare la decodare** – Predicțiile de tokeni sunt eșantionate din distribuția de probabilitate prezisă $p_i$, care este controlată de hiperparametrul *temperatura* $T$.



$T \ll 1$

$$p_i = \frac{\exp\left(\frac{x_i}{T}\right)}{\sum_{j=1}^{n} \exp\left(\frac{x_j}{T}\right)}$$

$T \gg 1$



*Remarcă: Temperaturile ridicate conduc la ieșiri mai creative, în timp ce temperaturile scăzute conduc la ieșiri mai deterministe.*

## 4 Antrenarea LLM-urilor

### 4.1 Preantrenare

❐ **Pierdere prin entropie încrucișată** – ***C**ross-**E**ntropy* (CE), adică *entropia încrucișată*, este o funcție de pierdere care penalizează modelul atunci când atribuie o probabilitate scăzută $p_i$ tokenului următor corect $i$:

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{V} y_i \log p_i$$

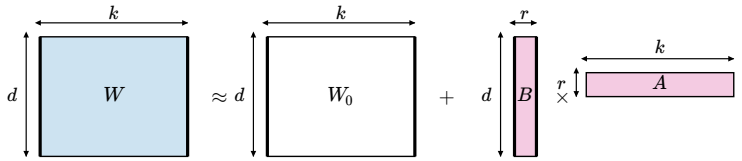
❐ **Legi de scalare** – *Legile de scalare* sunt legi empirice care descriu îmbunătățiri previzibile, de tip lege de putere, ale performanței modelului pe măsură ce cresc dimensiunea modelului, cantitatea de date de antrenare și puterea de calcul.

### 4.2 Ajustare fină

❐ **SFT** – ***S**upervised **F**ine**T**uning* (SFT), adică *ajustarea fină supervizată*, este o abordare de post-antrenare care aliniază comportamentul modelului cu o sarcină finală. Se bazează pe perechi intrare-ieșire de înaltă calitate, aliniate cu sarcina.

*Remarcă: Dacă datele SFT sunt despre instrucțiuni, acest pas se numește „ajustare pe instrucțiuni” (instruction tuning).*

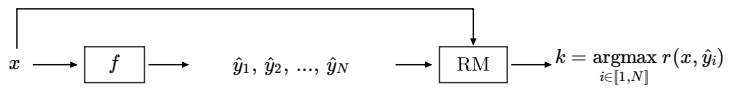
❐ **PEFT** – ***P**arameter-**E**fficient **F**ine**T**uning* (PEFT), adică *ajustarea fină eficientă ca număr de parametri*, este o categorie de metode folosite pentru a rula SFT eficient. În special, ***L**ow-**R**ank **A**daptation* (LoRA), adică adaptarea de rang mic, aproximează ponderile învățabile $W$ prin fixarea lui $W_0$ și învățarea, în schimb, a matricelor de rang mic $A, B$:



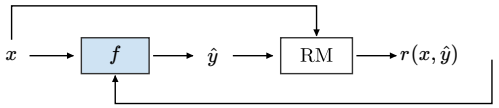

*Remarcă: Alte tehnici PEFT includ ajustarea prefixului (prefix tuning) și inserarea de straturi adaptor (adapter layers).*

### 4.3 Ajustarea preferințelor

❐ **Model de recompensă** – Un ***R**eward **M**odel* (RM), adică *model de recompensă*, este un model care prezice cât de bine se aliniază o ieșire $\hat{y}$ cu comportamentul dorit, dat fiind intrarea $x$. Eșantionarea ***B**est-of-**N*** (BoN), numită și *eșantionare prin respingere* (*rejection sampling*), este o metodă care folosește un model de recompensă pentru a selecta cel mai bun răspuns dintre $N$ generări.




❒ **Învățare prin întărire** – ***R**einforcement **L**earning* (RL), adică *învățarea prin întărire*, este o abordare care folosește RM și actualizează modelul $f$ pe baza recompenselor pentru ieșirile sale generate. Dacă RM se bazează pe preferințe umane, acest proces se numește ***R**einforcement **L**earning from **H**uman **F**eedback* (RLHF), adică învățare prin întărire din feedback uman.

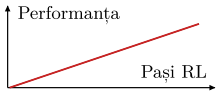
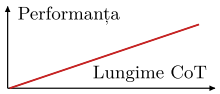



*Remarcă: Există și abordări supervizate, precum **D**irect **P**reference **O**ptimization (DPO), care combină RM și RL într-un singur pas supervizat.*

### 4.4 Raționament

❒ **Lanț de gândire** – ***C**hain-of-**T**hought* (CoT), adică *lanțul de gândire*, este un proces de raționament în care modelul descompune o problemă complexă într-o serie de pași intermediari. Aceasta îl ajută pe model să genereze răspunsul final corect.

❒ **Model de raționament** – Un *model de raționament* este un model care se bazează pe urme de raționament de tip CoT pentru a rezolva sarcini mai complexe de matematică, programare și logică. Pentru a-i spori capacitățile de raționament se folosesc două tipuri de metode de scalare:

| | Descriere | Ilustrație |
|---|---|---|
| **Scalare la antrenare** | Rulează RL mai mult timp pentru a lăsa modelul să învețe cum să producă urme de raționament de tip CoT înainte de a oferi un răspuns |   |
| **Scalare la inferență** | Lasă modelul să se gândească mai mult înainte de a oferi un răspuns, folosind cuvinte cheie de forțare a bugetului (*budget forcing*), precum `"Wait"` |   |

❒ **OPD** – ***O**n-**P**olicy **D**istillation* (OPD), adică *distilarea on-policy*, este o abordare de post-antrenare care se bazează pe traiectorii eșantionate din model și care folosește un profesor mai puternic pentru a oferi supraveghere la nivel de token pe acele traiectorii.

*Remarcă: Spre deosebire de recompensele rare, la nivel de secvență, OPD oferă un semnal de învățare dens, la nivel de token, din partea profesorului, pe tot parcursul traiectoriei generate.*

## 5 Învățarea prin întărire cu LLM-uri

### 5.1 Fundamente

❒ **Configurare** – În aplicațiile de RL cu LLM-uri:

- *Agentul* este modelul lingvistic, a cărui *politică* definește distribuția de probabilitate asupra acțiunilor de tip token următor.
- *Starea* este prompt-ul plus tokenii generați până în acel moment.
- *Acțiunea* este următorul token eșantionat.
- *Recompensa* evaluează răspunsul generat în funcție de comportamentul dorit sau de rezultatul sarcinii.

❒ **Gradienți de politică** – *Gradientul de politică* (*policy gradient*) este o familie de algoritmi RL care actualizează o politică pentru a favoriza acțiunile cu randamente eșantionate $R$ mai mari. Deoarece $R$ poate varia semnificativ între traiectorii, putem folosi o linie de bază $b(s)$ pentru a cuantifica acțiunile care performează mai bine decât se așteaptă, $R - b(s)$. Aceasta ajută la reducerea variației actualizării.

*Remarcă: REINFORCE este un exemplu de algoritm de gradient de politică.*

### 5.2 PPO

❒ **Funcția de valoare** – O *funcție de valoare* estimează recompensa viitoare așteptată pornind de la o stare dată sau de la o pereche stare-acțiune, în cadrul politicii curente.

❒ **Algoritm** – ***P**roximal **P**olicy **O**ptimization* (PPO), adică *optimizarea proximală a politicii*, este un algoritm RL popular care stimulează recompense mai mari, menținând totodată modelul aproape de modelul de bază, pentru a preveni exploatarea recompensei (*reward hacking*). Definind raportul de probabilitate al politicii $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, avem:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]$$

### 5.3 GRPO

❒ **Recompense verificabile** – *Recompensele verificabile* sunt recompense calculate automat pe baza unor rezultate verificabile în mod obiectiv, precum răspunsuri exacte, execuția codului sau demonstrații formale.

❒ **Algoritm** – ***G**roup **R**elative **P**olicy **O**ptimization* (GRPO), adică *optimizarea relativă de grup a politicii*, estimează avantajele politicii $\hat{A}_i$ comparând recompensele $R_i$ ale unui grup de $G$ răspunsuri eșantionate, evitând nevoia unui model de valoare separat. Definind $\hat{A}_i = \frac{R_i - \text{mean}(R_1,\ldots,R_G)}{\text{std}(R_1,\ldots,R_G)}$, avem:

$$\mathcal{L}_{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\mathbb{E}_t\left[\min\left(r_{i,t}(\theta)\hat{A}_i,\ \text{clip}\left(r_{i,t}(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_i\right) - \beta D_{\text{KL}}\left(\pi_\theta \| \pi_{\text{ref}}\right)\right]$$

*Remarcă: Există extensii care abordează limitările GRPO. De exemplu, Dr. GRPO corectează bias-ul legat de lungime, în timp ce DAPO îmbunătățește stabilitatea antrenării și explorarea prin ajustări ale clipping-ului și prin eșantionare dinamică.*

# 6 Sisteme LLM

## 6.1 Kernel-uri

□ **Flash attention** – *Flash attention* este o metodă exactă care optimizează calculele de atenție, folosind în mod ingenios hardware-ul GPU: utilizează memoria rapidă ***S**tatic **R**andom-**A**ccess **M**emory* (SRAM) pentru operațiile cu matrice, înainte de a scrie rezultatele în memoria mai lentă ***H**igh **B**andwidth **M**emory* (HBM).

*Remarcă: În practică, aceasta reduce consumul de memorie și accelerează calculele.*

□ **Paged attention** – *Paged attention* gestionează cache-ul KV în blocuri de memorie de dimensiune fixă, reducând fragmentarea memoriei și permițând partajarea eficientă a memoriei între secvențe.

## 6.2 Antrenare distribuită

□ **Paralelism** – *Paralelismul* distribuie antrenarea LLM-urilor pe mai multe dispozitive, prin partiționarea datelor, a calculului modelului sau a componentelor modelului.

□ **Tipuri** – Principalele forme de paralelism sunt:

- *Paralelism al datelor*: Replică modelul și împarte loturile de antrenare între dispozitive.
- *Paralelism tensorial*: Împarte calculele din interiorul straturilor individuale între dispozitive.
- *Paralelism pe etape* (*pipeline*): Împarte straturile modelului în etape plasate pe dispozitive diferite.
- *Paralelism al experților*: Distribuie experții MoE între dispozitive și direcționează tokenii în consecință.

*Remarcă: Antrenarea LLM-urilor la scară mare combină de obicei mai multe forme de paralelism.*

## 6.3 Optimizări ale inferenței

□ **Cache KV** – ***K**ey-**V**alue caching* (cache-ul cheie-valoare) stochează vectorii de cheie și de valoare calculați anterior în timpul decodării, evitând calculele de atenție redundante la generarea fiecărui token nou.

*Remarcă: Prompt caching reutilizează intrările din cache-ul KV pentru prefixele de prompt care se repetă.*

□ **Prefill vs decode** – Inferența LLM constă în două faze: *prefill*, care procesează toți tokenii prompt-ului în paralel pentru a inițializa cache-ul KV, și *decode*, care generează tokeni noi în mod autoregresiv, unul câte unul.

□ **Decodare speculativă** – *Decodarea speculativă* este o metodă care accelerează generarea, folosind un model *draft* (ciornă) mai mic pentru a propune mai mulți tokeni, care sunt apoi verificați în paralel de modelul țintă.

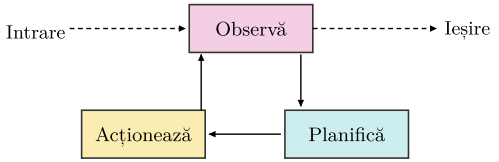
*Remarcă: Această metodă accelerează procesul de decodare deoarece propunerea de tokeni cu un model mic este ieftină, în timp ce modelul țintă poate verifica mai mulți tokeni propuși într-o singură trecere înainte (forward pass), în loc să îi genereze unul câte unul.*

# 7 Agenți AI

## 7.1 Privire de ansamblu

□ **Definiție** – Un *agent AI* este un sistem care urmărește în mod autonom obiective și duce la bun sfârșit sarcini în numele unui utilizator. Poate folosi diferite lanțuri de apeluri LLM pentru a face acest lucru.

□ **ReAct** – ***R**eason* + ***Act*** (ReAct), adică *raționează* + *acționează*, este un cadru care permite mai multe lanțuri de apeluri LLM pentru a duce la bun sfârșit sarcini complexe:




Acest cadru este compus din pașii de mai jos:

- *Observă*: Sintetizează acțiunile anterioare și afirmă explicit ceea ce se știe în prezent.
- *Planifică*: Detaliază ce sarcini trebuie realizate și ce instrumente trebuie apelate.
- *Acționează*: Efectuează o acțiune printr-un API sau caută informații relevante într-o bază de cunoștințe.

*Remarcă: Evaluarea unui sistem agentic este dificilă. Cu toate acestea, ea se poate face totuși atât la nivel de componentă, prin intrări-ieșiri locale, cât și la nivel de sistem, prin lanțuri de apeluri.*

□ **Harness** – *Harness*-ul (cadrul de execuție) este sistemul din jurul modelului care gestionează contextul, instrumentele, starea și bucla agentului.

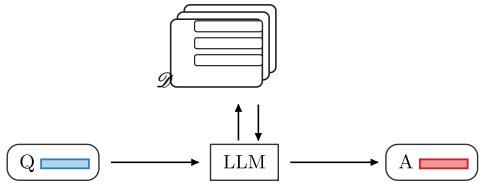
*Remarcă: Exemple includ Codex, Claude Code și Pi.*

## 7.2 Regăsire

□ **Retriever** – Un *retriever* (modul de regăsire) selectează documente relevante dintr-o bază de cunoștințe, folosind de obicei regăsire *sparse* (bazată pe cuvinte cheie), regăsire *dense* (bazată pe embedding-uri) sau o combinație hibridă a celor două.

□ **Reranker** – Un *reranker* (reordonator) reordonează documentele regăsite folosind un model de relevanță mai precis, îmbunătățind calitatea contextului transmis LLM-ului.

□ **RAG** – ***R**etrieval-**A**ugmented **G**eneration* (RAG), adică *generarea augmentată prin regăsire*, este o metodă care permite LLM-ului să acceseze cunoștințe externe relevante pentru a răspunde unei întrebări date. Este deosebit de utilă atunci când dorim să includem informații ulterioare datei limită de cunoștințe a LLM-ului preantrenat.



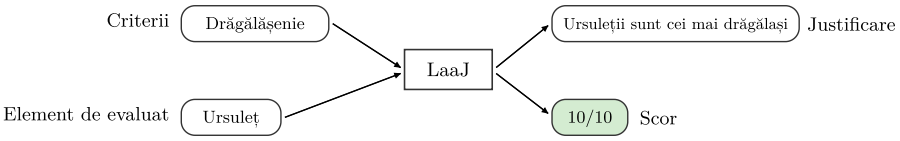

Dată fiind o bază de cunoștințe $\mathscr{D}$ și o întrebare, un modul de **R**egăsire aduce cele mai relevante documente, apoi **A**ugmentează prompt-ul cu informațiile relevante înainte de a **G**enera ieșirea.

*Remarcă: Calitatea RAG depinde în mare măsură de segmentarea documentelor (chunking), de calitatea regăsirii și de numărul de documente regăsite incluse în context.*

## 8 Evaluare

### 8.1 LLM-ca-Judecător

□ **LLM-ca-Judecător** – ***LLM**-as-a-**J**udge* (LaaJ), adică *LLM-ca-judecător*, este o metodă care folosește un LLM pentru a evalua ieșiri date pe baza unor criterii furnizate. În special, poate genera și o justificare pentru scorul acordat, ceea ce ajută la interpretabilitate.




Spre deosebire de metricile din era pre-LLM, precum ***R**ecall-**O**riented **U**nderstudy for **G**isting **E**valuation* (ROUGE), LaaJ nu are nevoie de niciun text de referință, ceea ce îl face convenabil pentru evaluarea oricărui tip de sarcină. În particular, LaaJ arată o corelație puternică cu evaluările umane atunci când se bazează pe un model mare și puternic (de ex. GPT-6), deoarece are nevoie de capacități de raționament pentru a funcționa bine.

*Remarcă: LaaJ este util pentru a efectua runde rapide de evaluare, dar este important să se monitorizeze alinierea dintre rezultatele LaaJ și evaluările umane, pentru a ne asigura că nu apare nicio divergență.*

□ **Biasuri frecvente** – Modelele LaaJ pot prezenta următoarele biasuri:

| | **Bias de poziție** | **Bias de verbozitate** | **Bias de auto-favorizare** |
|---|---|---|---|
| **Problemă** | Favorizează prima poziție în comparațiile pe perechi | Favorizează conținutul mai verbos | Favorizează ieșirile generate chiar de ele |
| **Soluție** | Media metricii pe poziții randomizate | Adaugă o penalizare pentru lungimea ieșirii | Folosește un judecător construit dintr-un alt model de bază |

*Remarcă: Lista de biasuri de mai sus nu este exhaustivă.*

### 8.2 Evaluarea agenților

□ **Componenta** – Evaluarea la nivel de *componentă* măsoară calitatea modulelor individuale ale agentului, precum regăsirea, selectarea instrumentelor sau executarea acțiunilor.

*Remarcă: Metricile cheie includ acuratețea, precizia, recall-ul, rata de succes a apelurilor de instrumente și latența.*

□ **Traiectorie** – Evaluarea la nivel de *traiectorie* evaluează întreaga secvență de observații, decizii și acțiuni întreprinse de un agent.

*Remarcă: Metricile cheie includ rata de succes a sarcinii, numărul de pași/apeluri de instrumente, rata de erori, latența și costul.*

## 9 LLM-uri de difuzie

### 9.1 Paradigma

□ **ARM** – Un ***A**uto**R**egressive **M**odel* (ARM), adică *model autoregresiv*, generează o secvență un token pe rând, condiționând fiecare token nou de toți tokenii generați anterior.

□ **LLM de difuzie** – Un *LLM de difuzie* (*diffusion LLM*, dLLM) generează text printr-un proces iterativ de eliminare a zgomotului (*denoising*) asupra întregii secvențe. Aceasta contrastează cu ARM-urile, care generează tokeni secvențial, de la stânga la dreapta. Cele două tipuri principale de dLLM sunt:

- *Difuzie continuă*: Elimină zgomotul din embedding-urile continue ale tokenilor, apoi asociază embedding-urile rezultate cu tokeni discreți.
- *Difuzie discretă*: Elimină zgomotul direct din tokeni, în spațiul discret.

*Remarcă: Difuzia discretă este în prezent cea mai comună formulare pentru LLM-urile de difuzie, în special prin difuzia mascată (masked diffusion), în care tokenii sunt mascați progresiv și reconstruiți.*

### 9.2 Antrenare

□ **Obiectiv** – Modelul este antrenat să reconstruiască tokenii originali dintr-o secvență coruptă, la un nivel de zgomot eșantionat aleatoriu.

□ **Proces direct** – *Procesul direct* (*forward process*) corupe progresiv textul curat, de ex. înlocuind o fracțiune tot mai mare de tokeni cu `[MASK]`.

### 9.3 Inferența

□ **Strategie** – La momentul inferenței, un LLM de difuzie pornește de la o secvență coruptă sau mascată și prezice repetat tokeni până când se obține un răspuns complet.

*Remarcă: LLM-urile de difuzie acceptă în mod natural generarea în ordine arbitrară și completarea de goluri (infilling), deoarece generarea nu este constrânsă să avanseze strict de la stânga la dreapta.*

□ **Program de decodare** – Un *program de decodare* (*decoding schedule*) stabilește care tokeni prezișI sunt acceptați sau remascați la fiecare pas de eliminare a zgomotului, echilibrând viteza și calitatea generării.